Python Exception Handling

`try` | `except` | `raise` | `print` | `else` | `finally`

PCAP-31-03 Certification Study Sheet

Cloud City Architects | Instructor: Shafan Sugarman

Three Types of Errors in Python

Before understanding exception handling, you need to know the three categories of errors and WHEN they happen.

Error Type	When It Happens	Example	Can try/except Fix It?
Syntax Error	BEFORE the program runs. Python can't parse the code.	`if x == 5` (missing colon)	**No.** Program never starts.
Runtime Error	DURING execution. Syntax is valid but operation fails.	`x = 10 / 0` (valid syntax, fails at runtime)	**Yes!** This is what exception handling is for.
Logical Error	NEVER crashes. Code runs fine but gives wrong results.	`avg = a + b / 2` (should be (a+b)/2)	**No.** Python doesn't know your intent.

Key Insight

Exception handling (try/except/raise) ONLY deals with runtime errors. It cannot fix syntax errors (program won't start) and cannot detect logical errors (Python doesn't know your intent). The PCAP exam tests whether you know which category an error falls into.

The Four Keywords: What Each One Does

Keyword	Purpose	Analogy	PCAP Weight
`try`	Wraps code that might fail at runtime	Putting on a seatbelt before driving	**Tested in every exception question**
`except`	Catches a specific type of error and handles it	The airbag deploying when you crash	**Tested in every exception question**
`raise`	Intentionally throws an exception you detected	Pulling the fire alarm when you see smoke	**Custom exceptions + validation**
`print`	Displays text to the user. Does NOT handle errors.	Yelling about the fire but not pulling the alarm	**Not part of exception handling**

Key Insight

print only TALKS. raise STOPS. If you detect bad data, print lets the program keep running with that bad data. raise forces execution to stop and someone upstream to deal with it.

The Complete Structure: try-except-else-finally

```
try:
    result = 10 / value

except ZeroDivisionError as e:
    # Catches division by zero
    print(f"Cannot divide by zero: {e}")

except TypeError as e:
    # Catches wrong types
    print(f"Wrong type: {e}")

else:
    # ONLY runs if NO exception occurred
    print(f"Success! Result: {result}")

finally:
    # ALWAYS runs, error or not
    print("Operation complete.")
```

Clause	When It Runs	Common Use
`try`	Always (first)	Wrap risky code — keep it as small as possible
`except`	Only if matching exception occurs. First match wins.	Handle specific error types. Order: specific → general.
`else`	Only if NO exception occurred in try	Success logic. Keeps try block minimal.
`finally`	ALWAYS runs — even after return statements	Close files, release resources, cleanup

Required order: try → except (one or more) → else (optional) → finally (optional). Any other order causes SyntaxError. You cannot have else without at least one except.

> **finally Overrides return**
> If both try and finally have return statements, finally's return WINS. The finally block runs before the function actually returns, and if finally returns a value, it replaces the try block's return value. This is a common PCAP exam trap.

Execution Flow: Step by Step

Step	What Happens	Explanation

Step 1	`try:`	Python enters the try block and starts running code inside it.
Step 2a	`Code succeeds`	No exception → skip all except blocks. Run else if present.
Step 2b	`Code fails or raise is called`	Exception created. Python immediately exits the try block.
Step 3	`except Type as e:`	Python searches except blocks top-to-bottom. First match wins.
Step 4	`Handler runs`	Code inside matching except executes. This is where print goes.
Step 5	`else:`	Only runs if Step 2a happened (no exception at all).
Step 6	`finally:`	Always runs regardless of outcome. Even after return statements.

Python Exception Hierarchy (PCAP Required)

All exceptions are organized in an inheritance tree. Understanding this hierarchy is critical because except clauses match by inheritance — a parent class catches ALL of its children.

```
BaseException
 ├── SystemExit
 ├── KeyboardInterrupt
 ├── GeneratorExit
 └── Exception
      ├── ArithmeticError
      │    ├── FloatingPointError
      │    ├── OverflowError
      │    └── ZeroDivisionError
      │
      ├── AssertionError
      ├── AttributeError
      ├── EOFError
      ├── ImportError
      │    └── ModuleNotFoundError
      │
      ├── LookupError
      │    ├── IndexError
      │    └── KeyError
      │
      ├── NameError
      │    └── UnboundLocalError
      │
      ├── OSError
      │    ├── ConnectionError
      │    │    ├── BrokenPipeError
      │    │    ├── ConnectionRefusedError
      │    │    └── ConnectionResetError
      │    ├── FileExistsError
      │    ├── FileNotFoundError
      │    ├── PermissionError
      │    └── TimeoutError
      │
      ├── RuntimeError
      │    ├── NotImplementedError
      │    └── RecursionError
      │
      ├── StopIteration
      ├── SyntaxError
      │    └── IndentationError
      │         └── TabError
      │
      ├── TypeError
      ├── ValueError
      │    └── UnicodeError
      │
      └── Warning
           ├── DeprecationWarning
           ├── RuntimeWarning
           └── UserWarning
```

Why the Hierarchy Matters for except

except clauses match by inheritance. A parent class catches ALL child exceptions.

except Clause	What It Catches
`except BaseException`	EVERYTHING — including SystemExit and KeyboardInterrupt
`except Exception`	Almost everything — but NOT SystemExit, KeyboardInterrupt, GeneratorExit
`except ArithmeticError`	ZeroDivisionError, OverflowError, FloatingPointError
`except LookupError`	IndexError, KeyError (any lookup failure)
`except OSError`	FileNotFoundError, PermissionError, ConnectionError, TimeoutError, etc.
`except ZeroDivisionError`	Only ZeroDivisionError — most specific

PCAP Trap: except Order with Inheritance

If except AppError comes before except ChildError, and ChildError inherits from AppError, the parent handler catches the child exception FIRST. The specific handler never runs. Always order from MOST SPECIFIC (child) to MOST GENERAL (parent). The exam loves testing this.

KeyboardInterrupt and except Exception:

KeyboardInterrupt inherits from BaseException, NOT Exception. So except Exception will NOT catch Ctrl+C. This is by design — you generally want users to be able to stop a program even when you're catching errors. A bare except: (no type) DOES catch KeyboardInterrupt, which is why bare except is discouraged.

When to Use Each Keyword

try/except: When Python Already Throws the Error

Code That Fails	Exception Thrown	Why
`float("abc")`	`ValueError`	Invalid literal for float
`10 / 0`	`ZeroDivisionError`	Division by zero
`my_list[999]`	`IndexError`	List index out of range
`my_dict["missing"]`	`KeyError`	Key not found
`int(None)`	`TypeError`	Wrong argument type
`open("nope.txt")`	`FileNotFoundError`	File does not exist
`int("3.14")`	`ValueError`	String is not valid integer literal

For all of these, just use **`try/except`** to catch them. Python throws these automatically — no need to raise.

Catching Multiple Exceptions: Tuple vs Separate

You can catch multiple exception types two ways. Both are valid, but they serve different purposes:

Same handler for both (tuple syntax):

```
try:
    hours = float(hours)
except (TypeError, ValueError) as e:
    # Parentheses required! Same response for both.
    print(f"Invalid input: {e}")
```

Different handler for each (separate blocks):

```
try:
    hours = float(hours)
except TypeError:
    # Handle wrong type specifically
    raise TypeError("Input must be a number, not " + type(hours).__name__)
except ValueError:
    # Handle wrong value specifically
    raise ValueError("Could not convert to number: " + str(hours))
```

PCAP Syntax Trap

except TypeError, ValueError: (without parentheses) is Python 2 syntax and causes a SyntaxError in Python 3. You MUST use parentheses: except (TypeError, ValueError):. This appears on almost every PCAP practice exam.

raise: When YOU Detect a Problem Python Doesn't Know About

Python has no idea about your business rules. It will happily calculate negative pay, process orders for 0 items, or register a user with age -5. These are YOUR rules, so YOU raise.

Your Check	You Raise	Why Python Won't
`hours < 0`	`NegativeValueError`	Payroll: hours must be positive
`age < 0 or age > 150`	`ValueError`	Business rule: realistic age
`balance < amount`	`InsufficientFundsError`	Banking: can't overdraw
`len(password) < 8`	`ValueError`	Security: password too short
`quantity == 0`	`ValueError`	Orders: must order 1+ items

The Rule

try/except = catching errors Python already throws. raise = throwing errors for YOUR rules that Python doesn't know about.

Why print Is NOT Error Handling

WRONG: Using print (bad data keeps flowing):

```
def calculate_pay(hours, rate):
    if hours < 0:
        print("Error: hours cannot be negative")
        # Function keeps running with hours = -5!
    return hours * rate  # Returns -100.00!
```

CORRECT: Using raise (bad data is stopped):

```
def calculate_pay(hours, rate):
    if hours < 0:
        raise NegativeValueError("Hours cannot be negative")
        # Function STOPS. This line never runs.
    return hours * rate
```

Then the caller catches it and prints:

```
try:
    pay = calculate_pay(-5, 20)
except NegativeValueError as e:
    print(f"Please fix your input: {e}")
    # Bad data never reached the calculation
```

> **The Order**
> raise stops the damage. except catches it. print tells the user. All three work together — print alone cannot do the stopping part.

Custom Exception Classes (PCAP Required)

Step 1: Define the class — it must inherit from Exception

```
class NegativeValueError(Exception):
    """Raised when negative values used where positive expected"""
    pass
```

Step 2: Raise it in your validation logic

```
def calculate_pay(hours, rate):
    if hours < 0 or rate < 0:
        raise NegativeValueError("Must be positive")
    return hours * rate
```

Step 3: Catch it in the caller

```
try:
    pay = calculate_pay(-5, 20)
except NegativeValueError as e:
    print(f"Validation error: {e}")
except ValueError as e:
    print(f"Type error: {e}")
```

The class is a blueprint. You don't 'access' it like a variable. You USE it in two places: raise to throw it, and except to catch it. If everything is in the same file, no import needed.

Software Layers and the Two-Level Pattern

In real software, code is organized in layers. Each layer has a different responsibility. Exceptions flow UPWARD through these layers until someone catches them.

The Layer Stack

Layer	Responsibility	Uses	Example
Layer 4: Top Handler	Catches everything. Decides what the user sees.	`try / except / print`	Web route, CLI main(), GUI event loop
Layer 3: Service Logic	Business rules. Catches data errors, raises business errors.	`try / except / raise`	process_payment(), run_payroll()
Layer 2: Validation	Checks data against YOUR rules.	`raise`	validate_age(), check_balance()
Layer 1: Utilities	Basic operations. Python's built-in exceptions happen here.	`raise (auto)`	float(), int(), file open()

Exceptions travel UP: Layer 1 throws → Layer 2 catches and re-throws → Layer 3 catches and re-throws → Layer 4 catches and responds. Nobody crashes because each layer handles what it can.

The Two-Level Pattern in Code

The function that finds the problem is NOT the function that decides what to do about it.

Low-level function: DETECTS and RAISES

```
def calculate_pay(hours, rate):
    """Low-level: validates and calculates. Raises on bad input."""
    try:
        hours = float(hours)
        rate = float(rate)
    except (TypeError, ValueError) as e:
        raise ValueError(f"Invalid input: {e}")

    if hours < 0 or rate < 0:
        raise NegativeValueError("Must be positive")

    if hours <= 40:
        return hours * rate
    else:
        return 40 * rate + (hours - 40) * rate * 1.5
```

High-level caller: CATCHES and DECIDES

```
def process_employee(name, hours, rate):
    """High-level: decides how to respond to errors."""
    try:
        pay = calculate_pay(hours, rate)
        print(f"{name}: ${pay:.2f}")
    except NegativeValueError as e:
        print(f"{name}: REJECTED - {e}")
    except ValueError as e:
        print(f"{name}: BAD INPUT - {e}")
```

Why This Separation Matters

Without Layers (Everything in One Place)	With Layers (Proper Separation)
The validation function prints errors, returns None, and the caller has to check for None everywhere.	The validation function raises. The caller catches. Each part does one job. No None-checking spaghetti.
If a web app, CLI, and batch job all call the function, they all get the same print output.	Web app returns JSON error. CLI prints to terminal. Batch job logs and skips. Same function, different responses.
print goes nowhere on a server — no terminal, no user watching.	raise travels up the stack to whoever IS listening — could be a web framework, an API, or a test suite.

Why return None Is Bad

Returning None hides the error. The caller gets None and might not check for it, causing subtle bugs downstream (like TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'). With raise, the caller is FORCED to handle the error — there's no silent failure.

Common Mistakes (PCAP Traps)

Wrong	Correct	Why It Matters
`except TypeError, ValueError:`	`except (TypeError, ValueError):`	Must use parentheses. Without them = SyntaxError in Python 3.
`Catching NegativeValueError before it's raised`	`Put raise inside try, or let the caller catch it`	You can only catch exceptions raised INSIDE the try block.
`except Exception: (first)`	`Put specific exceptions FIRST`	Parent catches all children. Specific handlers after it never run.
`Bare except: (no type)`	`Always specify the exception type`	Bare except catches SystemExit and KeyboardInterrupt too.
`return None in except`	`Let the exception propagate (raise)`	Returning None hides the error. Callers won't know something failed.
`print instead of raise`	`raise for validation, print in except`	print doesn't stop execution. Bad data continues flowing.
`Big try blocks`	`Wrap only the risky line`	Large try blocks can accidentally catch exceptions from unrelated code.
`Raising an exception inside finally`	`Avoid raising in finally blocks`	A new exception in finally replaces the original — you lose the real error.

Quick Decision Guide

Situation	Use	Why
float() might get a string	`try/except`	Python already throws ValueError
Dividing and denominator might be 0	`try/except`	Python already throws ZeroDivisionError
User enters negative hours	`raise`	Python doesn't know hours must be positive
Account balance too low for withdrawal	`raise`	Python doesn't know your business rules
Want to tell the user what went wrong	`print inside except`	Report AFTER catching, not instead of catching
File might not exist	`try/except`	Python throws FileNotFoundError
Password doesn't meet requirements	`raise`	Python doesn't know your password rules
Want to clean up resources	`finally`	Runs whether error occurred or not

Need to stop bad data from flowing	`raise`	print won't stop it. raise will.
Caller needs to decide how to respond	`raise`	Don't print in low-level functions

PCAP Exam Tips: Exception Handling

1. except blocks are checked top-to-bottom. First matching type wins. Put specific exceptions before general ones.
2. else only runs if the try block completed with NO exceptions. If any exception was raised (even if caught), else is skipped.
3. finally ALWAYS runs: after try succeeds, after except handles an error, even after return statements inside try or except.
4. If finally has a return statement, it OVERRIDES the return from try or except. This is a common exam gotcha.
5. raise with no arguments re-raises the current exception. Only valid inside an except block.
6. Multiple exceptions need parentheses: except (TypeError, ValueError): not except TypeError, ValueError:.
7. Custom exceptions must inherit from Exception (or a subclass of it).
8. except Exception catches almost everything. except BaseException catches even SystemExit and KeyboardInterrupt.
9. You cannot catch an exception raised OUTSIDE the try block. The raise must be inside try (or in a function called from inside try).
10. A parent except clause catches all child exceptions via inheritance. except LookupError catches both IndexError and KeyError.
11. int("3.14") raises ValueError, not TypeError. The type (string) is correct but the value cannot be converted.
12. Exceptions propagate UP the call stack. If inner() raises and doesn't catch, outer() gets a chance to catch it.
13. A new exception raised inside finally replaces the original exception. The first error is lost.
14. type(e).__name__ gives you the class name of the exception as a string (e.g., 'ZeroDivisionError').
15. The function that detects a problem should raise. The function that calls it should try/except. This is the two-level pattern.